

MI FB API

Version 3.0

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	07/27/2021
	- Modify Figure 1-1, the graphics layer supports overlaying to the SCL PATH, and added related instructions - In section 1.5, the chip difference part added the graphics layer overlay to the SCL PATH support in different chips - In Section 1.7, added INI parameters configuration	08/08/2021
	Added procfs introduction	08/25/2021
	Refine HW_LAYER_ID and Refine chip difference table	12/17/2021
	Added the difference table of Mochi	02/16/2022
	Added the difference table of Maroku	03/21/2022
	Added support for API "FBIO_WAITFORVSYNC"	04/11/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. OVERVIEW	1
1.1. Module Description	1
1.2. Basic Structure	1
1.3. Module Function	2
1.4. Application Scenario	3
1.5. Chip Difference	3
1.5.1 Taiyaki	4
1.5.2 Takoyaki	4
1.5.3 Ispahan	4
1.5.4 Pudding	4
1.5.5 Ikayaki	4
1.5.6 Tiramisu	4
1.5.7 Muffin	5
1.5.8 Mochi	5
1.5.9 Maruko	6
1.6. Principle	6
1.6.1 Graphics layer refresh mode	6
1.6.2 Relationship between virtual screen resolution, visible area and display resolution	7
1.7. Development Process	7
1.7.1 Compile Config	7
1.7.2 Configure MI_FB startup parameters	8
1.7.3 Interface call	10
1.8. Example	11
2. API REFERENCE	16
2.1. STANDARD FUNCTION	17
2.1.1 FBIOGET_VSCREENINFO	17
2.1.2 FBIOPUT_VSCREENINFO	18
2.1.3 FBIOGET_FSCREENINFO	19
2.1.4 FBIOPAN_DISPLAY	20
2.1.5 FBIO_WAITFORVSYNC	21
2.1.6 FBIOGETCMAP	22
2.1.7 FBIOPUTCMAP	22
2.2. EXTENDED FUNCTION	23
2.2.1 FBIOGET_SCREEN_LOCATION	23
2.2.2 FBIOSET_SCREEN_LOCATION	24
2.2.3 FBIOGET_SHOW	25
2.2.4 FBIOSET_SHOW	25
2.2.5 FBIOGET_GLOBAL_ALPHA	26
2.2.6 FBIOSET_GLOBAL_ALPHA	27
2.2.7 FBIOGET_COLORKEY	27
2.2.8 FBIOSET_COLORKEY	28

2.2.9	FBIOSET_DISPLAYLAYER_ATTRIBUTES.....	29
2.2.10	FBIOGET_DISPLAYLAYER_ATTRIBUTES.....	29
2.2.11	FBIOGET_CURSOR_ATTRIBUTE.....	30
2.2.12	FBIOSET_CURSOR_ATTRIBUTE.....	31
2.3.	MI_FB API.....	31
2.3.1	MI_FB_Open.....	31
2.3.2	MI_FB_Close.....	33
2.3.3	MI_FB_Mmap.....	34
2.3.4	MI_FB_Munmap.....	35
2.3.5	MI_FB_GetVarScreenInfo.....	35
2.3.6	MI_FB_SetVarScreenInfo.....	36
2.3.7	MI_FB_GetFixScreenInfo.....	37
2.3.8	MI_FB_PanDisplay.....	37
2.3.9	MI_FB_WaitForVsync.....	38
2.3.10	MI_FB_GetScreenLocation.....	39
2.3.11	MI_FB_SetScreenLocation.....	39
2.3.12	MI_FB_GetDisplayLayerAttr.....	40
2.3.13	MI_FB_SetDisplayLayerAttr.....	41
2.3.14	MI_FB_GetCursorAttr.....	41
2.3.15	MI_FB_SetCursorAttr.....	42
2.3.16	MI_FB_GetGlobalAlpha.....	43
2.3.17	MI_FB_SetGlobalAlpha.....	43
2.3.18	MI_FB_GetColorKey.....	44
2.3.19	MI_FB_SetColorKey.....	45
2.3.20	MI_FB_SetCmap.....	46
2.3.21	MI_FB_GetCmap.....	46
2.3.22	MI_FB_GetShow.....	47
2.3.23	MI_FB_SetShow.....	48
3.	FB DATA TYPE.....	49
3.1.	Standard Data Type.....	49
3.1.1	struct fb_bitfield.....	49
3.1.2	struct fb_var_screeninfo.....	50
3.1.3	struct fb_fix_screeninfo.....	53
3.1.4	struct fb_cmap_user.....	54
3.2.	MI_FB Data Type.....	55
3.2.1	MI_FB_BitField_t.....	55
3.2.2	MI_FB_VarScreenInfo_t.....	55
3.2.3	MI_FB_FixScreenInfo_t.....	55
3.2.4	MI_FB_ColorFmt_e.....	56
3.2.5	MI_FB_OuputColorSpace_e.....	56
3.2.6	MI_FB_DstDisplayPlane_e.....	57
3.2.7	MI_FB_DisplayLayerAttrMaskBit_e.....	58
3.2.8	MI_FB_CursorAttrMaskBit_e.....	59
3.2.9	MI_FB_DisplayLayerAttr_t.....	59

3.2.10	MI_FB_GlobalAlpha_t.....	61
3.2.11	MI_FB_ColorKey_t.....	62
3.2.12	MI_FB_CursorImage_t.....	62
3.2.13	MI_FB_CursorAttr_t.....	63
3.2.14	MI_FB_Rectangle_t.....	64
3.2.15	MI_FB_Cmap_t.....	64
4.	ERROR CODE.....	65
5.	PROCFS INTRODUCTION.....	66
5.1.	cat.....	66
5.2.	echo.....	70

1. OVERVIEW

1.1. Module Description

MI_FB is a graphics layer device driver based on Linux Framebuff. It abstracts the display buffer, shields the bottom-level difference of the image hardware, and allows the upper-level APP to directly read and write the display buffer in the graphics mode. The specific location and storage method of the physical display buffer is completed by MI_FB driver. In addition to providing basic functions of Linux Framebuff, MI_FB also provides extended functions such as Alpha between layers, ColorKey, and display area setting.

1.2. Basic Structure

MI_FB manages multiple graphics layers. With chip support, each graphics layer can be overlaid on different video processing/output paths.

1. The video preview overlay path means that the graphics layer can be overlaid on the video display unit which is managed by MI_DISP submodule. If the graphics layer is overlaid on the video display unit, you need to configure MI_DISP submodule first, and then operate MI_FB submodule to draw the graphics layer, finally, the video display unit outputs the overlaid video data to the display device, and the video display unit determines the final display resolution.
2. Video processing overlay path means that the graphics layer can be overlaid on the video processing unit which is managed by MI_SCL submodule. If the graphics layer is overlaid on the video processing unit, you need to configure MI_SCL submodule, and then operate MI_FB submodule to draw the graphics layer, finally, the video processing unit outputs the overlaid video data to the video encoding unit or to the video display unit for preview playback.

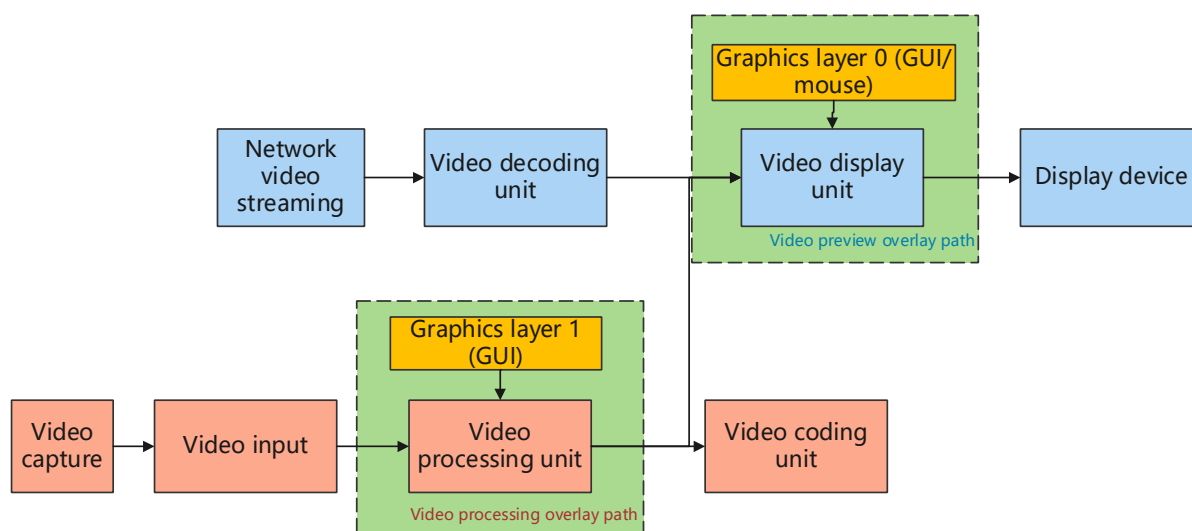


Figure 1-1 Graphics layer overlay path

From the perspective of application development, there are two ways to draw the graphics layer based on MI_FB.

Method 1: APP uses MI_FB based on Linux file system.

Method 2: Develop APP based on the API provided by MI_FB.

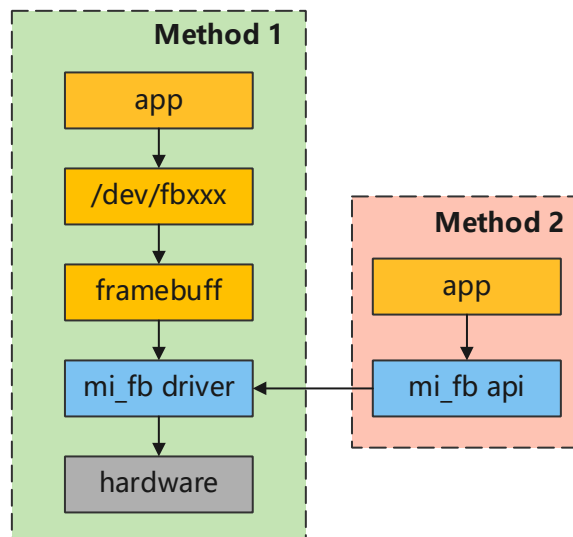


Figure 1-2: MI_FB usage

Note: The two methods are mutually exclusive, only one of them can be selected for development

1.3. Module Function

Linux Framebuff standard functions supported by MI_FB:

1. Get fixed info of display device (such as video memory size)
2. Map (or unmap) physical video memory to virtual memory space
3. Operate physical video memory like ordinary files
4. Set hardware display resolution and pixel format
5. Read, write, display and other operations from any location in the physical video memory
6. Support setting and obtaining 256-color palette

Extended functions added by MI_FB:

1. Set and get the Alpha value of the overlay graphics layer
2. Set and get the ColorKey value of the overlay graphics layer
3. Set the display position of the current overlay graphics layer (the offset relative to the origin of the screen)
4. Set the display area of the current overlay graphics layer (supports canvas enlargement display)
5. Set and get the display status of the current overlay (show/hide)
6. Manage the size of physical video memory and the number of overlay graphics layers through INI config file
7. Dynamically synchronized display device output resolution
8. Support the operation of the mouse graphics layer

Linux Framebuff standard functions not supported by MI_FB:

1. Set and get the Linux Framebuff corresponding to the console

2. Get real-time info of hardware scanning
3. Get hardware related info
4. Set hardware sync timing
5. Set hardware sync signal mechanism

1.4. Application Scenario

1. Pure Linux

Framebuffer-based applications in Linux (such as MiniGUI) can also be developed based on the API interface provided by MI_FB.

2. Pure rtos

The standard Linux Framebuff framework is not supported under RTOS. Applications can develop UI based on the API interface provided by MI_FB (such as using lightweight lvgl to develop UI).

3. Dualos

In Dualos, applications running on Linux can be developed based on the standard Framebuff framework or MI_FB API; applications running on RTOS can only be developed based on MI_FB API.

1.5. Chip Difference

MI_FB has two forms of user interface. MI_FB adds the concept of FB ID to keep user behavior consistent. The following figure shows the relationship between FB ID and Framebuff device node.

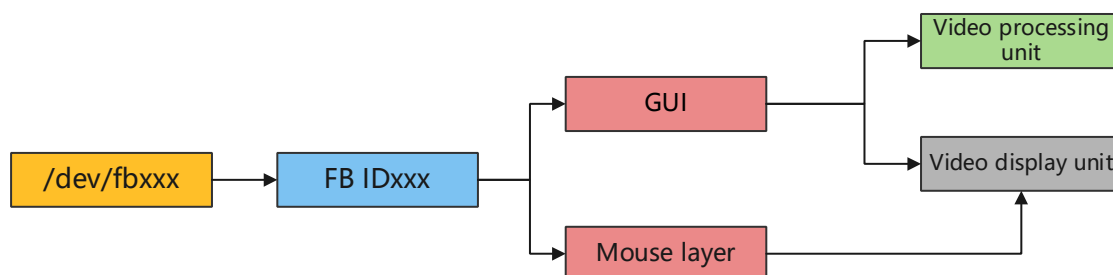


Figure 1-3: FB ID and device node relationship

MI_FB defines the ID growth sequence to be consistent with the device node of Framebuff, that is, /dev/fbxxx corresponds to FB IDxxx.

Each FB ID of MI_FB controls at most one GUI and one mouse layer, which means that the application can use the same FB ID to operate the GUI and mouse layer bound to the ID at the same time, GUI can be superimposed on video display unit or video processing unit, the mouse layer can only be superimposed on video display unit, and the superimposed output path of GUI and the mouse layer is configured in config.ini. [Section 1.7.2](#) describes in detail how to configure the INI.

MI_FB can manage 16 FB IDs. The specific supported IDs are related to the chip capabilities. The differences between different chips are shown in the following table:

- ※ The Layer ID corresponds to FB_HWLAYER_ID items in the MI_FB startup parameters, please refer to the Layer ID when configuring the startup parameters.
- ※ The hierarchical relationship between different graphics layers in the same video display unit with the same PATH is GUI -> OSD -> CURSOR from bottom up.

※ Alpha0/1 indicates the actual Alpha value when the color format is ARGB1555 and the Alpha bit is 0/1. On some graphics layers, alpha1 is only supported and Alpha 0 is not supported, which is marked N.

1.5.1 Taiyaki

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	0	N	Y	Y	N	N	N	Y	N	Y

1.5.2 Takoyaki

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	0	N	Y	Y	N	N	N	Y	N	Y

1.5.3 Ispahan

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video processing unit		GUI	NA	N	Y	Y	N	N	N	Y	N	Y

1.5.4 Pudding

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video processing unit		GUI	NA	N	Y	Y	N	N	N	Y	N	Y

1.5.5 Ikayaki

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N

1.5.6 Tiramisu

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	0	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH1	GUI	3	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	2	N	Y	Y	N	N	N	Y	N	Y
Video processing unit	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

1.5.7 Muffin

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	0	N	Y	Y	N	N	N	Y	N	Y
		OSD	2	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH1	GUI	9	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	8	N	Y	Y	N	N	N	Y	N	Y
		OSD	10	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH2	GUI	17	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	16	N	Y	Y	N	N	N	Y	N	Y
		OSD	18	N	Y	Y	N	N	N	Y	N	Y
Video processing unit	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ The video display unit of the Muffin series of chips has added an OSD graphics layer, typically as the default graphics layer that MI_RGN overlaid to DISP, and if MI_FB wants to use the OSD graphics layer, MI_RGN cannot overlay the OSD to DISP.

1.5.8 Mochi

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video	DISP	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N

display unit	PATH0	CURSOR	0	N	Y	Y	N	N	N	Y	N	Y
		OSD	2	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH1	GUI	9	Y	Y	Y	Y	N	N	Y	Y	N
		CURSOR	8	N	Y	Y	N	N	N	Y	N	Y
		OSD	10	N	Y	Y	N	N	N	Y	N	Y
Video processing unit	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ The video display unit of the Mochi series of chips has added an OSD graphics layer, typically as the default graphics layer that MI_RGN overlaid to DISP, and if MI_FB wants to use the OSD graphics layer, MI_RGN cannot overlay the OSD to DISP.

1.5.9 Maruko

Graphics layer			Layer ID	Color Format							Scale Up	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
Video display unit	DISP PATH0	GUI	2	Y	Y	Y	Y	N	N	Y	N	Y
Video processing unit	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ Video display unit GUI layer, video processing unit GUI and SCL/DISP GOP in MI_RGN share the same hardware. FB cannot be used when MI_RGN superimposes OSD on SCL/DISP. And vice versa.

1.6. Principle

1.6.1 Graphics layer refresh mode

1. Single buff mode

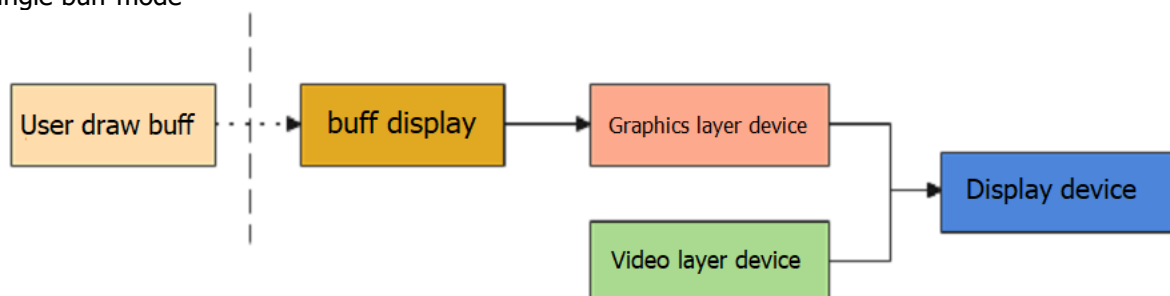


Figure 1-4: Single buff refresh mode

In this mode, MI_FB provides a display buff, and app draw directly, which can save memory, but user will see the drawing process of the graphics on the display device. You can also apply for another buff for graphics drawing, and then move it to display buff, but this method will produce jagged.

2. Double buff mode

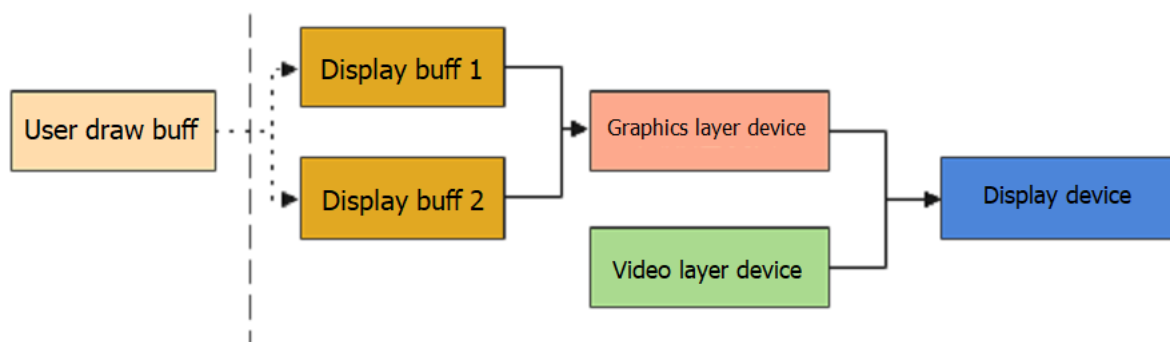


Figure 1-5: Double buff refresh mode

In this mode, MI_FB provides two display buffs. When the graphics layer device is using display buff2, you can draw on buff1, and then switch to the starting position of buff1 by setting the virtual screen offset position. It consumes more memory, but the display effect is the best.

1.6.2 Relationship between virtual screen resolution, visible area and display resolution

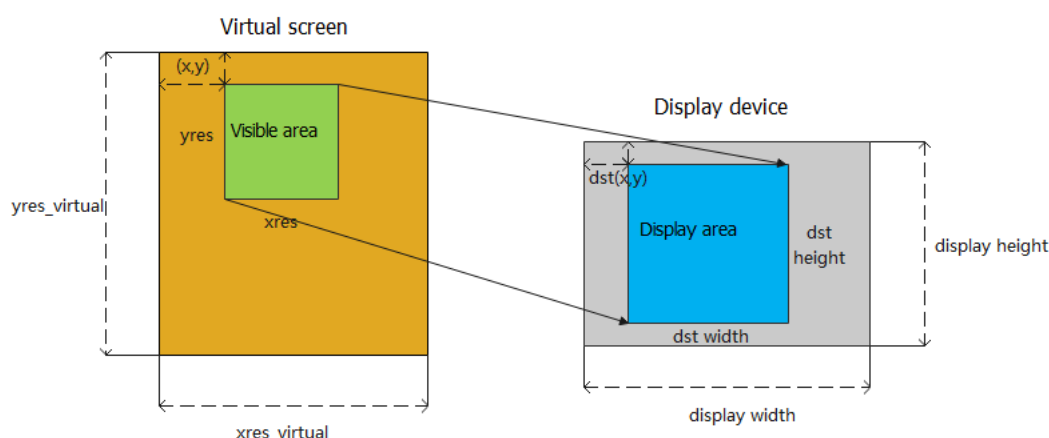


Figure 1-6: Relationship between resolutions

The virtual screen size can be regarded as the video memory size provided by MI_FB. You can define the resolution of the visible area (xres, yres) in the config file. MI_FB will adjust the virtual screen resolution (xres_virtual, yres_virtual) according to the range of the visible area. The default screen resolution is $xres_virtual = xoffset + xres$, xoffset is the offset of the visible area in the horizontal direction of the virtual screen, and xres is width.

MI_FB adjusts yres_virtual according to xres_virtual, pixel format and video memory size; you can freely change the visible area (xoffset, yoffset) within the range of (xres_virtual, yres_virtual) to switch the display content, and the double-buff refresh mode is also implemented based on this.

The graphics layer device reads the memory data from the visible area, and then outputs it to the display device according to the display area attributes configured by user. The display area attributes include the display area range and the offset position relative to the origin of the display screen.

The process from the visible area to the display area supports magnification, that is, the visible area (xres, yres) $<$ display area (dst_width, dst_height) $\leq$ display device resolution (display width, display height), and the display device resolution is decided by output MI_DISP Timing.

1.7. Development Process

1.7.1 Compile Config

1. Enter alkaid project root directory, make menuconfig

```
Misc Options --->
Rtos --->
Sdk Config --->
```

2. Press Enter to enter the Sdk Config sub-option

```
Verify Compile Config --->
Interface Compile Config --->
Misc Compile Config --->
Mhal Compile Config --->
```

3. Press Enter to enter the Interface Compile Config sub-option

```
[ ] wlan
[ ] dummy
[*] fb
```

4. Press Space bar to select the fb submodule, and recompile the project

After compilation, mi_fb.ko will generated in project/release, and release mi_fb.h and mi_fb_datatype.h to this directory at the same time. It will be packaged into images by default in pure Linux. But in dualos, it will not be packaged by default and needs to be manually compiled and installed.

1.7.2 Configure MI_FB startup parameters

MI_FB startup parameters are managed in config.ini.

m_fbdevList defined in the ROOTsection of config.ini describes the entry of GUI or mouse layer parameters of the current platform.

1. FB_DEVICE describes the configuration information superimposed on the GUI of the video display unit;
2. FB_VIDEO and FB_BUF_ID0_PATH describe the configuration information superimposed on GUI of the video processing unit;
3. FB_CURSOR describes the configuration information of the mouse layer, the mouse layer will be superimposed on GUI.

```
[ROOT]
m_fbdevList = {\
    FB_DEVICE,\
    FB_CURSOR,\
    FB_VIDEO,\
}

[FB_DEVICE]
FB_HWLAYER_ID = 1;
FB_HWWIN_ID = 0;
FB_HWWIN_FORMAT = 5;
FB_HWLAYER_OUTPUTCOLOR = 0;
FB_WIDTH = 800;
FB_HEIGHT = 480;
```

```

FB_TIMING_WIDTH = 1920;
FB_TIMING_HEIGHT = 1080;
FB_MMAP_NAME = E_MMAP_ID_FB;
FB_BUFFER_LEN = 8192
#unit:Kbyte, total size = FB_BUFFER_LEN*1024

[FB_CURSOR]
FB_HWLAYER_ID = 0;
FB_HWWIN_ID = 0;
FB_HWWIN_FORMAT = 6;
FB_HWLAYER_OUTPUTCOLOR = 0;
FB_MMAP_NAME = E_MMAP_ID_HW_CURSOR;

[FB_VIDEO]
FB_BUF_ID = 0
FB_BUF_CNT = 3
FB_HWWIN_FORMAT = 6
FB_WIDTH = 1920
FB_HEIGHT = 1080
FB_TIMMING_WIDTH = 1920
FB_TIMMING_HEIGHT = 1080

[FB_BUF_ID0_PATH]
FB_MOD = 34
FB_DEV = 1
FB_CHN = 0
FB_PORT = 0
FB_GWIN_ID = 7

```

Note:

1. Adding multiple GUI layers (if supported by the chip) needs to add the new section name to m_fbdevList for management, otherwise it will not take effect. For example: the current INI configuration has a section of FB_DEVICE describing the information of a GUI layer, you can add a section of FB_DEVICE1 to describe another GUI layer, add the section name of FB_DEVICE1 to m_fbdevList.
2. The growth order of /dev/fbxxx device nodes is consistent with the order in m_fbdevList.

The description of each configuration parameter in config.ini is as follows:

Parameter	Description
FB_HWLAYER_ID	The current graphics layer ID
FB_HWWIN_ID	Hardware device ID (determine which output PATH of the video layer device DISP is superimposed on)
FB_HWWIN_FORMAT	Describe the canvas pixel format
FB_HWLAYER_OUTPUTCOLOR	Describe the color space (YUV/RGB) output by the current overlay

Parameter	Description
FB_WIDTH	Describe the visible area range (xres)
FB_HEIGHT	Describe the visible area range (yres)
FB_TIMING_WIDTH	Describe the display device resolution (display width)
FB_TIMING_HEIGHT	Describe the display device resolution (display height)
FB_MMAP_NAME	Video memory application related parameters (default is E_MMAP_ID_FB)
FB_BUFFER_LEN	Describe the video memory size (unit: Kbyte)
FB_BUF_ID	Associate the FB_BUF_ID0_PATH field
FB_BUF_CNT	Calculate the video memory size of the graphics layer superimposed on the video processing unit
FB_MOD	Superimpose the graphics layer MODULE ID of video processing unit
FB_DEV	Superimpose the graphics layer DEV ID of video processing unit
FB_CHN	Superimpose the graphics layer CHN ID of video processing unit
FB_PORT	Superimpose the graphics layer PORT ID of video processing unit

1.7.3 Interface call

There are two types of interface calls:

1. File system call based on standard Linux Framebuff;
2. Interface call based on MI_FB API.

Both of them are achieving the same function.

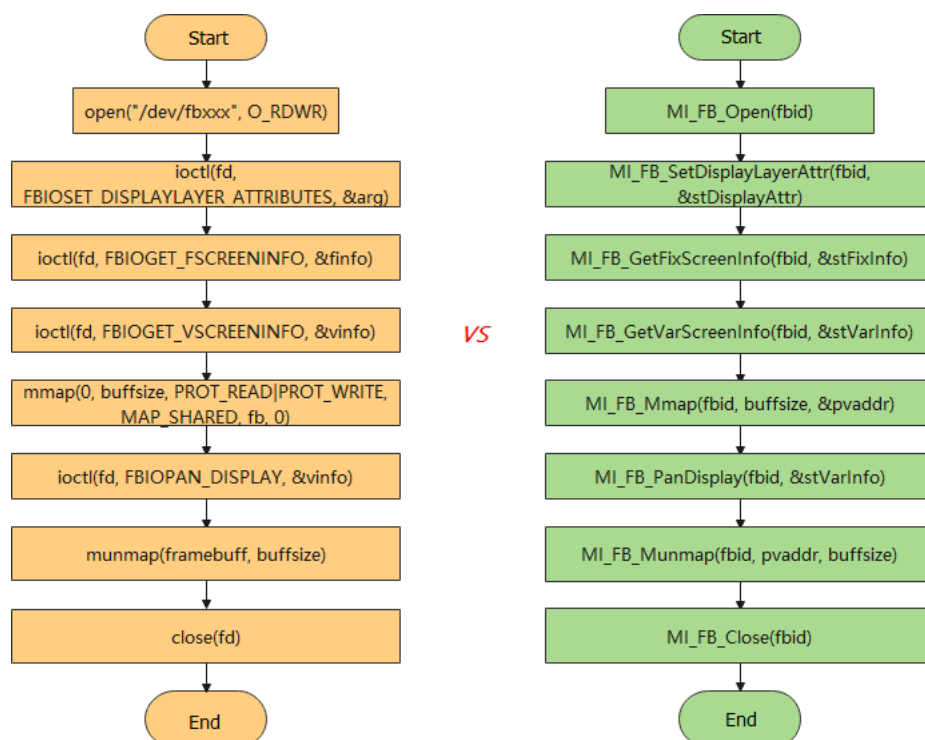


Figure 1-7: MI_FB interface call process

Steps:

1. Open the specified FB device
2. Set pixel format, visible area, etc.
3. Get video memory info allocated to MI_FB
4. Map physical video memory to virtual memory space
5. Call ioctl or MI_FB API to set ColorKey, inter-layer Alpha, display position and area, etc
6. Operate the virtual memory address to complete the specific drawing task
7. Unmap video memory
8. Close the FB device
9. Exit

1.8. Example

This example is based on Linux Framebuff and MI_FB API to display three color blocks arranged in steps of red, orange and blue in the middle of the screen, using the ARGB8888 pixel format.

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/fb.h>
#include "mi_sys.h"
#include "mi_disp.h"
#include "mi_disp_datatype.h"
#include "mi_fb.h"
#include "mi_fb_datatype.h"

static void *framebuffer = NULL;
#define LINUX_FRAMEBUFFER

void set_disp_dev(void)
{
    MI_DISP_PubAttr_t stPubAttr;
    memset(&stPubAttr, 0, sizeof(MI_DISP_PubAttr_t));
    stPubAttr.u32BgColor = YUYV_BLACK;
    stPubAttr.eIntfType == E_MI_DISP_INTF_TTL;
    MI_DISP_SetPubAttr(0, &stPubAttr);
    MI_PANEL_Init(E_MI_DISP_INTF_TTL);
}

#ifdef LINUX_FRAMEBUFFER
// Based on Linux Framebuff
void drawRect_rgb32 (int x0, int y0, int width, int height, int color, struct fb_var_screeninfo *vinfo, struct fb_fix_screeninfo
```

```
*finfo, char *fb)
{
    const int bytesPerPixel = 4;
    const int stride = finfo->line_length / bytesPerPixel;

    int *dest = (int *) (fb)
        + (y0 + vinfo->yoffset) * stride + (x0 + vinfo->xoffset);

    int x, y;
    for (y = 0; y < height; ++y)
    {
        for (x = 0; x < width; ++x)
        {
            dest[x] = color;
        }
        dest += stride;
    }
}

int main(int argc, char **argv)
{
    const char *devfile = "/dev/fb0";
    int fbFd = 0;
    long int screensize = 0;

    MI_SYS_Init(0);

    //set disp pub
    set_disp_dev();
    fbFd = open(devfile, O_RDWR);
    if(fbFd == -1)
    {
        perror("Error: cannot open framebuffer device");
        exit(1);
    }
    //get fb_fix_screeninfo
    if(ioctl(fbFd, FBIOGET_FSCREENINFO, &finfo) == -1)
    {
        perror("Error reading fixed information");
        exit(2);
    }
    //get fb_var_screeninfo
    if(ioctl(fbFd, FBIOGET_VSCREENINFO, &vinfo) == -1)
    {

```

```

    perror("Error reading variable information");
    exit(3);
}
screensize = finfo.smem_len;

/* Map the device to memory */
framebuffer = (char *)mmap(0, screensize, PROT_READ | PROT_WRITE, MAP_SHARED,
    fbFd, 0);
if(framebuffer == MAP_FAILED)
{
    perror("Error: Failed to map framebuffer device to memory");
    exit(4);
}
//draw red rectangle
drawRect_rgb32 (vinfo.xres / 8, vinfo.yres / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xffff0000, &vinfo, &finfo, framebuffer);

//draw DarkGoldenrod rectangle
drawRect_rgb32 (vinfo.xres * 3 / 8, vinfo.yres * 5 / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xffb8860b, &vinfo, &finfo, framebuffer);

//draw blue rectanble
drawRect_rgb32 (vinfo.xres * 5 / 8, vinfo.yres * 9 / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xff0000ff, &vinfo, &finfo, framebuffer);

munmap(framebuffer, screensize);
framebuffer = NULL;

close(fbFd);

return 0;
}
#else
//Based on MI_FB_API
void drawRect_rgb32(int x0, int y0, int width, int height, int color, MI_FB_VarScreenInfo_t *vinfo, MI_FB_FixScreenInfo_t
*finfo, char *fb)
{
    const int bytesPerPixel = 4;
    const int stride = finfo->u32LineLength/ bytesPerPixel;

    int *dest = (int *) (fb
        + (y0 + vinfo->u32Yoffset) * stride + (x0 + vinfo->u32Xoffset);

    int x, y;

```

```
for (y = 0; y < height; ++y)
{
    for (x = 0; x < width; ++x)
    {
        dest[x] = color;
    }
    dest += stride;
}

int main(int argc, char **argv)
{
    MI_S32 s32Ret = 0;
    MI_U8 u8FbId = 0;
    MI_U32 screensize = 0;

    MI_SYS_Init(0);

    //set disp pub
    set_disp_dev();

    s32Ret = MI_FB_Open(u8FbId);
    if(s32Ret != MI_SUCCESS)
    {
        printf("open fb id:%d failed\n", u8FbId);
        exit(1);
    }
    s32Ret = MI_FB_GetFixScreenInfo(u8FbId, &stFixScreenInfo);
    if(s32Ret != MI_SUCCESS)
    {
        printf("get fix screen info failed\n");
        exit(2);
    }

    s32Ret = MI_FB_GetVarScreenInfo(u8FbId, &stVarScreenInfo);
    if(s32Ret != MI_SUCCESS)
    {
        printf("get var screen info failed\n");
        exit(3);
    }
    screensize = stFixScreenInfo.u32SmemLen;
    MI_FB_Mmap(u8FbId, screensize, &framebuffer);
    if(!framebuffer)
    {
```

```
    printf("mmap framebuffer failed\n");
    exit(4);
}
//draw red rectangle
drawRect_rgb32 (stVarScreenInfo.u32Xres / 8, stVarScreenInfo.u32Yres / 8,
    stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xffff0000, &stVarScreenInfo, &stFixScreenInfo,
framebuffer);

//draw DarkGoldenrod rectangle
drawRect_rgb32 (stVarScreenInfo.u32Xres * 3 / 8, stVarScreenInfo.u32Yres * 3 / 8,
    stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xffb8860b, &stVarScreenInfo, &stFixScreenInfo,
framebuffer);

//draw blue rectanble
drawRect_rgb32 (stVarScreenInfo.u32Xres * 5 / 8, stVarScreenInfo.u32Yres * 5 / 8,
    stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xff0000ff, &stVarScreenInfo, &stFixScreenInfo,
framebuffer);

MI_FB_Munmap(u8FbId, framebuffer, screensize);
framebuffer = NULL;

MI_FB_Close(u8FbId);
return 0;
}
#endif
```

2. API REFERENCE

Two forms of MI_FB user mode interface:

1. File operations based on Linux Framebuff, embodied in the form of ioctl.

int ioctl(int fd, unsigned long cmd, CMD_DATA_TYPE *cmddata);

CMD_DATA_TYPE changes with the parameter cmd, the cmd supported by MI_FB is as follows.

Parameter	Description	Input/Output
fd	The descriptor of Framebuffer device file, which is the return value of calling the open function.	Input
cmd	The main cmds are as follows: • FBIOGET_VSCREENINFO: Get screen variable info • FBIOPUT_VSCREENINFO: Set screen variable info • FBIOGET_FSCREENINFO: Get screen fixed info • FBIOPAN_DISPLAY: Set PAN display • FBIOGETCMAP: Get color table info • FBIOPUTCMAP: Set color table info • FBIOGET_SCREEN_LOCATION: Get the overlay display area • FBIOSET_SCREEN_LOCATION: Set the overlay display area • FBIOGET_SHOW: Get the overlay display status • FBIOSET_SHOW: Set the overlay display status • FBIOGET_GLOBAL_ALPHA: Get the overlay Alpha attribute • FBIOSET_GLOBAL_ALPHA: Set the overlay Alpha attribute • FBIOGET_COLORKEY: Get the overlay Colorkey attribute • FBIOSET_COLORKEY: Set the overlay Colorkey attribute • FBIOGET_DISPLAYLAYER_ATTRIBUTES: Get overlay attribute • FBIOSET_DISPLAYLAYER_ATTRIBUTES: Set overlay attribute • FBIOGET_CURSOR_ATTRIBUTE: Get mouse layer attribute • FBIOSET_CURSOR_ATTRIBUTE: Set mouse layer attribute	Input
cmddata	Data type corresponding to cmd: • Get or set screen variable info: struct fb_var_screeninfo *type • Get screen fixed info: struct fb_fix_screeninfo *type • Set PAN display: struct fb_var_screeninfo *type • Get or set the origin of the screen overlay coordinate: MI_FB_Rectangle_t *type • Get or set the overlay display status: MI_BOOL *type • Get or set the overlay Alpha: MI_FB_GlobalAlpha_t *type • Get or set the overlay Colorkey: MI_FB_ColorKey_t *type • Get or set the overlay attribute: MI_FB_DisplayLayerAttr_t *type • Get or set mouse layer attribute: MI_FB_CursorAttr_t *type	Input/Output

2. Based on MI_FB API in the form of dynamic library or static library, the following APIs are currently supported.

API	Function
MI_FB_Open	Open FB device
MI_FB_Close	Close FB device
MI_FB_Mmap	Map physical video memory to virtual memory space
MI_FB_Munmap	Unmap physical video memory to virtual memory space
MI_FB_GetVarScreenInfo	Get screen variable info
MI_FB_SetVarScreenInfo	Set screen variable info
MI_FB_GetFixScreenInfo	Get FB fixed info
MI_FB_PanDisplay	Set the display to start at a different offset in the virtual resolution
MI_FB_GetScreenLocation	Get the display area of the overlay on screen
MI_FB_SetScreenLocation	Set the display area of the overlay on screen
MI_FB_GetDisplayLayerAttr	Get layer info
MI_FB_SetDisplayLayerAttr	Set layer info
MI_FB_GetCursorAttr	Get mouse layer info
MI_FB_SetCursorAttr	Set mouse layer info
MI_FB_GetGlobalAlpha	Get overlay Alpha settings
MI_FB_SetGlobalAlpha	Set overlay Alpha
MI_FB_GetColorKey	Get overlay colorkey settings
MI_FB_SetColorKey	Set overlay colorkey
MI_FB_SetCmap	Set the color table
MI_FB_GetCmap	Get the color table
MI_FB_GetShow	Get overlay display status
MI_FB_SetShow	Show or hide the overlay

Section 1.4 introduces the use of two forms of API in different scenarios, you can choose to use one of them according to the actual situation.

Note: Only one of the two calling methods can be selected.

2.1. STANDARD FUNCTION

2.1.1 FBIOGET_VSCREENINFO

➤ Function

Get screen variable information

➤ Syntax

```
int ioctl (int fd, FBIOGET\_VSCREENINFO, struct fb_var_screeninfo *var);
```

➤ Description

Use this interface to get screen variable information, including primarily resolution and pixel format.

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOGET_VSCREENINFO	ioctl cmd	Input
var	Variable information structure pointer	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: fb.h

➤ Related Interface

[FBIOPUT_VSCREENINFO](#)

2.1.2 FBIOPUT_VSCREENINFO

➤ Function

Set Framebuffer screen resolution and pixel format, etc.

➤ Syntax

```
int ioctl (int fd, FBIOPUT\_VSCREENINFO, struct fb_var_screeninfo *var);
```

➤ Description

Use this interface to set screen resolution and pixel format

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOPUT_VSCREENINFO	ioctl number	Input
var	Variable information structure pointer	Input

➤ Return Value

- 0: Successful.
- -1: Failed.

- Requirement
 - Header file: fb.h
- Note

The actual resolution and offset must be guaranteed to be within the virtual resolution range. Otherwise, system will automatically adjust the actual resolution to keep it within the coverage of the virtual resolution range.
- Related Interface

[FBIOGET_VSCREENINFO](#)

2.1.3 FBIOGET_FSCREENINFO

- Function

Get Framebuffer fixed information
- Syntax


```
int ioctl (int fd, FBIOGET\_FSCREENINFO, struct fb\_fix\_screeninfo *fix);
```
- Description

Use this interface to get Framebuffer fixed information, including memory start physical address, memory size and line spacing, etc.
- Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOGET_FSCREENINFO	ioctl cmd	Input
fix	Fixed information structure pointer	Output
- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: fb.h
- Note

N/A.
- Related Interface

N/A.

2.1.4 FBIOPAN_DISPLAY

➤ Function

Set to display from different offsets in the virtual resolution

➤ Syntax

```
int ioctl (int fd, FBIOPAN\_DISPLAY, struct fb\_var\_screeninfo *var);
```

➤ Description

Use this interface to set the display position from different offsets in the virtual resolution; the actual resolution remains unchanged. As shown in the figure below, (xres_virtual, yres_virtual) means the virtual resolution, (xres, yres) the resolution actually shown, and (xoffset, yoffset) the offset used.

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOPAN_DISPLAY	ioctl cmd	Input
var	Variable information structure pointer	Input

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: fb.h

➤ Note

The actual resolution and offset must be guaranteed to be within the virtual resolution range. Otherwise, the setting will fail.

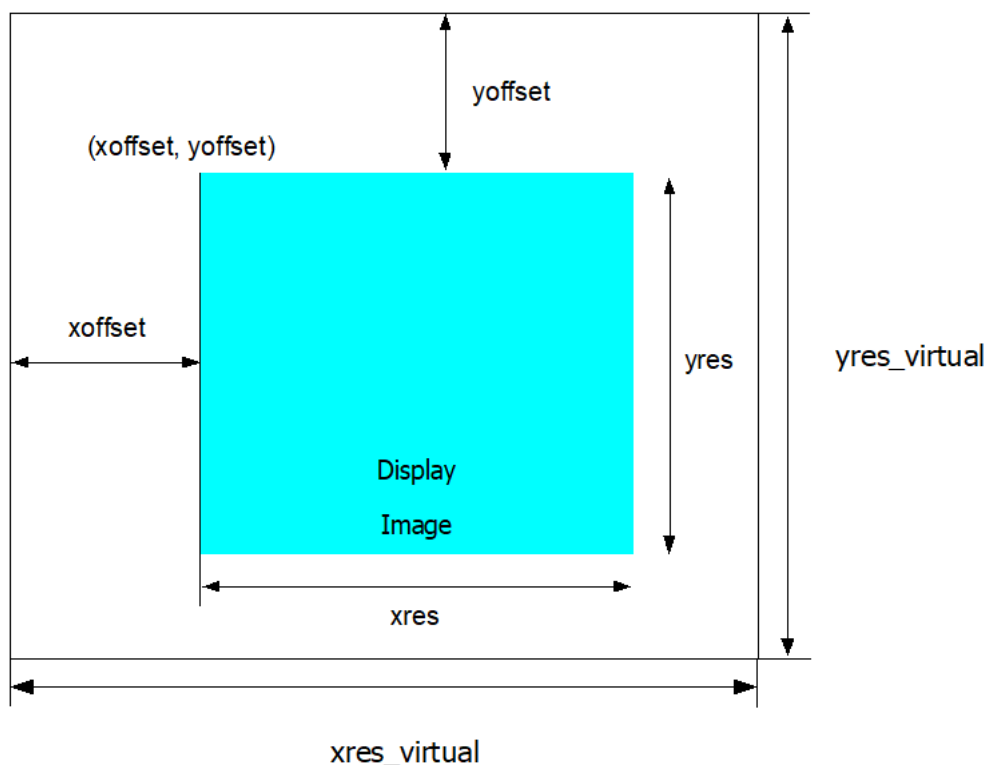


Figure 2-8: Setting the Display Position from Different Offsets in the Virtual Resolution

2.1.5 FBIO_WAITFORVSYNC

➤ Function

Wait for a signal that screen vertical refresh is complete(Vsync).

➤ Syntax

```
int ioctl (int fd, FBIO\_WAITFORVSYNC, u32 *zero);
```

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOGETCMAP	ioctl cmd	Input
zero	be sure to pass argument value 0 or it will not work.	Input

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: fb.h

2.1.6 FBIOGETCMAP

➤ Function

Get the color table info.

➤ Syntax

```
int ioctl (int fd, FBIOGETCMAP, struct fb_cmap_user *cmap);
```

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOGETCMAP	ioctl cmd	Input
cmap	Describe the color table info	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: fb.h

➤ Note

NA

➤ Related Interface

[FBIOPUTCMAP](#)
[MI_FB_SetCmap](#)

2.1.7 FBIOPUTCMAP

➤ Function

Set the color table info.

➤ Syntax

```
int ioctl (int fd, FBIOPUTCMAP, struct fb_cmap_user *cmap);
```

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOPUTCMAP	ioctl cmd	Input
cmap	Describe the color table info	Input

- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: fb.h
- Note

NA
- Related Interface

[FBIOGETCMAP](#)
[MI_FB_GetCmap](#)

2.2. EXTENDED FUNCTION

2.2.1 FBIOGET_SCREEN_LOCATION

- Function

Get overlay screen display region
- Syntax


```
int ioctl (int fd, FBIOGET\_SCREEN\_LOCATION, MI_FB_Rectangle_t* pstRectangle);
```
- Description

Use this interface to get overlay screen display region

- Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOGET_SCREEN_LOCATION	ioctl cmd	Input
pstRectangle	Pointer to MI_FB_Rectangle_t structure, including (x,y) coordinates, width, and height. It refers to the overlay screen display region.	Output

- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement

- Header file: `mi_fb_datatype.h`

➤ Related Interface

[FBIOSET_SCREEN_LOCATION](#)

2.2.2 FBIOSET_SCREEN_LOCATION

➤ Function

Set overlay screen display region

➤ Syntax

```
int ioctl (int fd, FBIOSET\_SCREEN\_LOCATION, MI_FB_Rectangle_t* pstRectangle);
```

➤ Description

Use this interface to set overlay screen display region

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSET_SCREEN_LOCATION	ioctl cmd	Input
pstRectangle	Pointer to MI_FB_Rectangle_t structure, including (x, y) coordinates, width, and height. It refers to the overlay screen display region.	Input

➤ Return Value

- 0: Successful.
- 1: Failed.

➤ Requirement

- Header file: `mi_fb_datatype.h`

➤ Example

N/A.

➤ Note

N/A.

➤ Related Interface

[FBIOGET_SCREEN_LOCATION](#)

2.2.3 FBIOGET_SHOW

➤ Function

Get current overlay display status

➤ Syntax

```
int ioctl (int fd, FBIOGET\_SHOW, MI_BOOL *bShow);
```

➤ Description

Use this interface to get current overlay display status

➤ Parameter

Parameter Name	Description	Input/Output
Fd	Framebuffer device file descriptor	Input
FBIOGET_SHOW	ioctl cmd	Input
bShow	Indicate the current overlay status: *bShow = MS_TRUE: Current overlay is in Show status *bShow = MS_FALSE: Current overlay is in Hidden status	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mi_fb_datatype.h

➤ Related Interface

[FBIOSET_SHOW](#)

2.2.4 FBIOSET_SHOW

➤ Function

Show or hide the overlay

➤ Syntax

```
int ioctl (int fd, FBIOSET\_SHOW, MI_BOOL *bShow);
```

➤ Description

Use this interface to set the overlay display status: show or hide

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSSET_SHOW	ioctl cmd	Input
bShow	Display status of the overlay: *bShow = MS_TRUE: Show the current overlay *bShow = MS_FALSE: Hide the current overlay	Input

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mi_fb_datatype.h

➤ Related Interface

[FBIOSGET_SHOW](#)

2.2.5 FBIOSGET_GLOBAL_ALPHA

➤ Function

Get overlay Alpha setting

➤ Syntax

```
int ioctl (int fd, FBIOSGET\_GLOBAL\_ALPHA, MI\_FB\_GlobalAlpha\_t *pstAlpha);
```

➤ Description

Use this interface to get current overlay Alpha setting

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSGET_GLOBAL_ALPHA	ioctl cmd	Input
pstAlpha	Pointer to MI_FB_GlobalAlpha_t structure	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mi_fb_datatype.h

➤ Related Interface

[FBIOSET_GLOBAL_ALPHA](#)

2.2.6 FBIOSET_GLOBAL_ALPHA

➤ Function

Set overlay Alpha

➤ Syntax

```
int ioctl (int fd, FBIOSET\_GLOBAL\_ALPHA, MI\_FB\_GlobalAlpha\_t *pstAlpha);
```

➤ Description

Use this interface to set current overlay Alpha function

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSET_GLOBAL_ALPHA	ioctl cmd	Input
pstAlpha	Pointer to MI_FB_GlobalAlpha_t structure	Input

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mi_fb_datatype.h

➤ Related Interface

[FBIOGET_GLOBAL_ALPHA](#)

2.2.7 FBIOGET_COLORKEY

➤ Function

Get overlay colorkey

➤ Syntax

```
int ioctl (int fd, FBIOGET\_COLORKEY, MI\_FB\_ColorKey\_t* pstColorKey);
```

➤ Description

Use this interface to get overlay colorkey

➤ Parameter

Parameter Name	Description	Input/Output
Fd	Framebuffer device file descriptor	Input
FBIOGET_COLORKEY	ioctl cmd	Input
pstColorKey	Pointer to MI_FB_ColorKey_t structure	Output

- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: mi_fb_datatype.h
- Related Interface
 - [FBIOSET_COLORKEY](#)

2.2.8 FBIOSET_COLORKEY

- Function

Set the overlay colorkey
- Syntax


```
int ioctl (int fd, FBIOSET\_COLORKEY, MI_FB_ColorKey_t* pstColorKey);
```
- Description

Use this interface to set current overlay colorkey function

- Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSET_COLORKEY	ioctl cmd	Input
pstColorKey	Pointer to MI_FB_ColorKey_t structure	Input

- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: mi_fb_datatype.h
- Related Interface
 - [FBIOGET_COLORKEY](#)

2.2.9 FBIOSET_DISPLAYLAYER_ATTRIBUTES

➤ Function

Set graphic layer information

➤ Syntax

```
int ioctl (int fd, FBIOSET\_DISPLAYLAYER\_ATTRIBUTES,
           MI_FB_DisplayLayerAttr_t* pstLayerInfo);
```

➤ Description

This interface is used to set graphic layer information, including screen display position, canvas resolution, memory resolution, screen display resolution, and capability of pre-multiplication of the graphic layer. For more details, see description with respect to MI_FB_DisplayLayerAttr_t.

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSET_DISPLAYLAYER_ATTRIBUTES	ioctl cmd	Input
pstLayerInfo	Pointer to MI_FB_DisplayLayerAttr_t structure	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mi_fb_datatype.h

➤ Related Interface

[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#)

2.2.10 FBIOGET_DISPLAYLAYER_ATTRIBUTES

➤ Function

Get graphic layer information

➤ Syntax

```
int ioctl (int fd,
           FBIOGET\_DISPLAYLAYER\_ATTRIBUTES,
           MI_FB_DisplayLayerAttr_t * pstLayerInfo);
```

➤ Description

Used to get graphic layer information, including screen display position, memory resolution, screen display resolution, and capability of pre-multiplication of the graphic layer.

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
F BIOGET_DISPLAYLAYER_ATTRIBUTES	ioctl cmd	Input
pstLayerInfo	Pointer to graphic layer information structure	Output

➤ Return Value

- 0: Successful.
- -1: Failed.

➤ Requirement

- Header file: mstarFb.h

➤ Related Interface

[F BIOGET_DISPLAYLAYER_ATTRIBUTES](#)

2.2.11 F BIOGET_CURSOR_ATTRIBUTE

➤ Function

Get cursor graphic layer information

➤ Syntax

```
int ioctl (int fd,
           F BIOGET\_CURSOR\_ATTRIBUTE,
           MI\_FB\_CursorAttr\_t *pstCursorAttr)
```

➤ Description

Get cursor graphic layer information, including hotspot, position, Alpha, Colorkey, and visibility.

➤ Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
F BIOGET_CURSOR_ATTRIBUTE	ioctl cmd	Input
pstCursorAttr	Pointer to MI_FB_CursorAttr_t structure	Output

- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: `mi_fb_datatype.h`
- Related Interface
 - [FBIOSET_CURSOR_ATTRIBUTE](#)

2.2.12 FBIOSET_CURSOR_ATTRIBUTE

- Function

Set cursor graphic layer information
- Syntax


```
int ioctl (int fd,
            FBIOSET\_CURSOR\_ATTRIBUTE,
            MI\_FB\_CursorAttr\_t *pstCursorAttr)
```
- Description

Set cursor graphic layer information, including hotspot, position, Alpha, Colorkey, and visibility
- Parameter

Parameter Name	Description	Input/Output
fd	Framebuffer device file descriptor	Input
FBIOSET_CURSOR_ATTRIBUTE	ioctl cmd	Input
pstCursorAttr	Pointer to MI_FB_CursorAttr_t structure	Output
- Return Value
 - 0: Successful.
 - -1: Failed.
- Requirement
 - Header file: `mi_fb_datatype.h`
- Related Interface
 - [FBIOGET_CURSOR_ATTRIBUTE](#)

2.3. MI_FB_API

2.3.1 MI_FB_Open

➤ Function

The function is the same as open framebuffer device node.

➤ Syntax

MI_S32 [MI_FB_Open](#) (MI_U32 u32FbId);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB device number	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h
- Library file:libmi_fb.a

➤ Example

```
MI_S32 s32Ret = MI_SUCCESS;
MI_U32 u32FbId = 0;
void *framebuff = NULL;
MI_U32 u32ScreenSize;
MI_FB_VarScreenInfo_t stVarScreenInfo;
MI_FB_FixScreenInfo_t stFixScreenInfo;

memset(&stVarScreenInfo, 0, sizeof(MI_FB_VarScreenInfo_t));
memset(&stFixScreenInfo, 0, sizeof(MI_FB_FixScreenInfo_t));

s32Ret = MI_FB_Open(u32FbId);
if(MI_SUCCESS == s32Ret)
{
    MI_FB_GetFixScreenInfo(u32FbId, &stFixScreenInfo);
    MI_FB_GetVarScreenInfo(u32FbId, &stVarScreenInfo);
    u32ScreenSize = stFixScreenInfo.u32SmemLen;
    if(u32ScreenSize)
        MI_FB_Mmap(u32FbId, u32ScreenSize, &framebuff);
    if(framebuff)
    {
        draw(x, y, framebuff);
    }
}
```

```
    stVarScreenInfo.u32Xoffset = 0;  
    stVarScreenInfo.u32Yoffset = stVarScreenInfo.u32Yres;  
    MI_FB_PanDisplay(u32FbId, &stVarScreenInfo);  
}  
MI_FB_Close(u32FbId);  
}
```

➤ Related Interface

[MI_FB_Close](#)

2.3.2 MI_FB_Close

➤ Function

The function is the same as close framebuffer device node.

➤ Syntax

MI_S32 [MI_FB_Close](#) (MI_U32 u32FbId);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_Open](#)

2.3.3 MI_FB_Mmap

➤ Function

The function is the same as framebuffer mmap.

➤ Syntax

MI_S32 [MI_FB_Mmap](#) (MI_U32 u32FbId, MI_U32 u32size, void **pVirAddr);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
u32size	buff size	Input
pVirAddr	Return the mapped virtual address	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h
- Library file:libmi_fb.a

➤ Example

NA

- Related Interface
NA

2.3.4 MI_FB_Munmap

- Function
The function is the same as framebuffer munmap.
- Syntax
MI_S32 [MI_FB_Munmap](#) (MI_U32 u32FbId, void *pVirAddr, MI_U32 u32Size);
- Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pVirAddr	Mapped virtual address	Input
u32Size	buff size	Input

- Return Value
 - 0: Successful.
 - Non-zero, failure, please refer to [error code](#) for details
- Requirement
 - Header file:mi_fb.h
 - Library file:libmi_fb.a
- Example
NA
- Related Interface
NA

2.3.5 MI_FB_GetVarScreenInfo

- Function
The function is the same as [FBIOGET_VSCREENINFO](#).
- Syntax
MI_S32 [MI_FB_GetVarScreenInfo](#) (MI_U32 u32FbId, [MI_FB_VarScreenInfo_t](#) *pstVarScreenInfo);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstVarScreenInfo	Describe screen variable info	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_SetVarScreenInfo](#)
[FBIOGET_VSCREENINFO](#)

2.3.6 MI_FB_SetVarScreenInfo

➤ Function

The function is the same as [FBIOPUT_VSCREENINFO](#).

➤ Syntax

MI_S32 [MI_FB_SetVarScreenInfo](#) (MI_U32 u32FbId, [MI_FB_VarScreenInfo_t](#) *pstVarScreenInfo);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstVarScreenInfo	Describe screen variable info	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetVarScreenInfo](#)

[FBIOPUT_VSCREENINFO](#)

2.3.7 MI_FB_GetFixScreenInfo

➤ Function

The function is the same as [FBIOGET_FSCREENINFO](#).

➤ Syntax

```
MI_S32 MI_FB_GetFixScreenInfo (MI_U32 u32FbId, MI_FB_FixScreenInfo_t
*pstFixScreenInfo);
```

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstFixScreenInfo	Describe screen fixed info	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[FBIOGET_FSCREENINFO](#)

2.3.8 MI_FB_PanDisplay

➤ Function

The function is the same as [FBIOPAN_DISPLAY](#).

➤ Syntax

```
MI_S32 MI_FB_PanDisplay (MI_U32 u32FbId, MI_FB_VarScreenInfo_t *pstVarScreenInfo);
```

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstVarScreenInfo	Describe screen variable info	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

※ Note

NA

➤ Example

NA

➤ Related Interface

[FBIOPAN_DISPLAY](#)

2.3.9 MI_FB_WaitForVsync

➤ Function

The function is the same as [FBIO_WAITFORVSYNC](#).

➤ Syntax

MI_S32 [MI_FB_WaitForVsync](#) (MI_U32 u32FbId);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

2.3.10 MI_FB_GetScreenLocation

➤ Function

The function is the same as [FBIOGET_SCREEN_LOCATION](#).

➤ Syntax

MI_S32 [MI_FB_GetScreenLocation](#) (MI_U32 u32FbId, [MI_FB_Rectangle_t](#) *pstWinRect);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstWinRect	Describe the display area of the overlay on screen	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_SetScreenLocation](#)
[FBIOGET_SCREEN_LOCATION](#)

2.3.11 MI_FB_SetScreenLocation

➤ Function

The function is the same as [FBIOSET_SCREEN_LOCATION](#).

➤ Syntax

MI_S32 [MI_FB_SetScreenLocation](#) (MI_U32 u32FbId, [MI_FB_Rectangle_t](#) *pstWinRect);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstWinRect	Describe the display area of the overlay on screen	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

- Requirement
 - Header file:mi_fb.h mi_fb_datatype.h
 - Library file:libmi_fb.a

- Example
NA

- Related Interface
 - [MI_FB_GetScreenLocation](#)
 - [FBIOSET_SCREEN_LOCATION](#)

2.3.12 MI_FB_GetDisplayLayerAttr

- Function
The function is the same as [FBIOGET_DISPLAYLAYER_ATTRIBUTES](#).

- Syntax
MI_S32 [MI_FB_GetDisplayLayerAttr](#) (MI_U32 u32FbId, MI_FB_DisplayLayerAttr_t *pstDisplayLayerAttr);

- Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstDisplayLayerAttr	Describe the layer info	Output

- Return Value
 - 0: Successful.
 - Non-zero, failure, please refer to [error code](#) for details

- Requirement
 - Header file:mi_fb.h mi_fb_datatype.h
 - Library file:libmi_fb.a

- Example
NA

- Related Interface
 - [MI_FB_SetDisplayLayerAttr](#)
 - [FBIOGET_DISPLAYLAYER_ATTRIBUTES](#)

2.3.13 MI_FB_SetDisplayLayerAttr

➤ Function

The function is the same as [FBIOSET_DISPLAYLAYER_ATTRIBUTES](#).

➤ Syntax

MI_S32 [MI_FB_SetDisplayLayerAttr](#) (MI_U32 u32FbId, [MI_FB_DisplayLayerAttr_t](#) *pstDisplayLayerAttr);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstDisplayLayerAttr	Describe the layer info	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetDisplayLayerAttr](#)
[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#)

2.3.14 MI_FB_GetCursorAttr

➤ Function

The function is the same as [FBIOGET_CURSOR_ATTRIBUTE](#).

➤ Syntax

MI_S32 [MI_FB_GetCursorAttr](#) (MI_U32 u32FbId, [MI_FB_CursorAttr_t](#) *pstCursorAttr);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstCursorAttr	Describe the mouse layer info	Output

➤ Return Value

- 0: Successful.

- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_SetCursorAttr](#)
[FBIOGET_CURSOR_ATTRIBUTE](#)

2.3.15 MI_FB_SetCursorAttr

➤ Function

The function is the same as [FBIOSET_CURSOR_ATTRIBUTE](#).

➤ Syntax

MI_S32 [MI_FB_SetCursorAttr](#) (MI_U32 u32FbId, [MI_FB_CursorAttr_t](#) *pstCursorAttr);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstCursorAttr	Describe the mouse layer info	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetCursorAttr](#)
[FBIOSET_CURSOR_ATTRIBUTE](#)

2.3.16 MI_FB_GetGlobalAlpha

➤ Function

The function is the same as [FBIOGET_GLOBAL_ALPHA](#).

➤ Syntax

MI_S32 [MI_FB_GetGlobalAlpha](#) (MI_U32 u32FbId, [MI_FB_GlobalAlpha_t](#) *pstAlpha);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstAlpha	Describe Alpha of the overlay	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_SetGlobalAlpha](#)
[FBIOSET_GLOBAL_ALPHA](#)

2.3.17 MI_FB_SetGlobalAlpha

➤ Function

The function is the same as [FBIOSET_GLOBAL_ALPHA](#).

➤ Syntax

MI_S32 [MI_FB_SetGlobalAlpha](#) (MI_U32 u32FbId, [MI_FB_GlobalAlpha_t](#) *pstAlpha);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstAlpha	Describe Alpha of the overlay	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

- Requirement
 - Header file:mi_fb.h mi_fb_datatype.h
 - Library file:libmi_fb.a

- Example

NA

- Related Interface
 - [MI_FB_GetGlobalAlpha](#)
 - [FBIOSET_GLOBAL_ALPHA](#)

2.3.18 MI_FB_GetColorKey

- Function

The function is the same as [FBIOGET_COLORKEY](#).
- Syntax

MI_S32 [MI_FB_GetColorKey](#) (MI_U32 u32FbId, [MI_FB_ColorKey_t](#) *pstClrKey);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstClrKey	Describe colorkey info of the overlay	Output

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_SetColorKey](#)
[FBIOGET_COLORKEY](#)

2.3.19 MI_FB_SetColorKey

➤ Function

The function is the same as [FBIOSET_COLORKEY](#).

➤ Syntax

MI_S32 [MI_FB_SetColorKey](#) (MI_U32 u32FbId, [MI_FB_ColorKey_t](#) *pstClrKey);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstClrKey	Describe colorkey info of the overlay	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetColorKey](#)
[FBIOSET_COLORKEY](#)

2.3.20 MI_FB_SetCmap

➤ Function

The function is the same as [FBIOPUTCMAP](#).

➤ Syntax

MI_S32 [MI_FB_SetCmap](#) (MI_U32 u32FbId, [MI_FB_Cmap_t](#) *pstCmap);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstCmap	Describe the color table info	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h mi_fb_datatype.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetCmap](#)

2.3.21 MI_FB_GetCmap

➤ Function

The function is the same as FBIOGUTCMAP.

➤ Syntax

MI_S32 [MI_FB_GetCmap](#) (MI_U32 u32FbId, [MI_FB_Cmap_t](#) *pstCmap);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pstCmap	Describe the color table info	Output

- Return Value
 - 0: Successful.
 - Non-zero, failure, please refer to [error code](#) for details
- Requirement
 - Header file:mi_fb.h mi_fb_datatype.h
 - Library file:libmi_fb.a
- Example
NA
- Related Interface
[MI_FB_SetCmap](#)

2.3.22 MI_FB_GetShow

- Function
The function is the same as [FBIOGET_SHOW](#).
- Syntax
MI_S32 [MI_FB_GetShow](#) (MI_U32 u32FbId, MI_BOOL *pbShown);
- Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
pbShown	Overlay display status	Output

- Return Value
 - 0: Successful.
 - Non-zero, failure, please refer to [error code](#) for details
- Requirement
 - Header file:mi_fb.h
 - Library file:libmi_fb.a
- Example
NA
- Related Interface

[MI_FB_SetShow](#)
[FBIOGET_SHOW](#)

2.3.23 MI_FB_SetShow

➤ Function

The function is the same as [FBIOSET_SHOW](#).

➤ Syntax

MI_S32 [MI_FB_SetShow](#) (MI_U32 u32FbId, MI_BOOL bShown);

➤ Parameter

Parameter	Description	Input/Output
u32FbId	FB DEVICE NUMBER	Input
bShown	1: show 0: hide	Input

➤ Return Value

- 0: Successful.
- Non-zero, failure, please refer to [error code](#) for details

➤ Requirement

- Header file:mi_fb.h
- Library file:libmi_fb.a

➤ Example

NA

➤ Related Interface

[MI_FB_GetShow](#)
[FBIOSET_SHOW](#)

3. FB DATA TYPE

Data types defined in the Linux Framebuff standard:

Data type	Description
struct fb_bitfield	Bit field information for setting pixel format
struct fb_var_screeninfo	Variable screen information
struct fb_fix_screeninfo	Fixed screen information
struct fb_cmap_user	Describe the color table info

Data types defined in MI_FB:

Data type	Description
MI_FB_BitField_t	Bit field information for setting pixel format
MI_FB_VarScreenInfo_t	Variable screen information
MI_FB_FixScreenInfo_t	Fixed screen information
MI_FB_ColorFmt_e	Collection of pixel formats supported by SigmaStar FB
MI_FB_OuputColorSpace_e	Color space output by FB overlay
MI_FB_DstDisplayPlane_e	The target module that FB overlay output to display chain
MI_FB_DisplayLayerAttrMaskBit_e	Specify the attributes to be updated in MI_FB_DisplayLayerAttr_t
MI_FB_CursorAttrMaskBit_e	Specify the attributes to be updated in MI_FB_CursorAttr_t
MI_FB_DisplayLayerAttr_t	Describe overlay attribute info
MI_FB_GlobalAlpha_t	Alpha information structure
MI_FB_ColorKey_t	Colorkey information structure
MI_FB_CursorImage_t	Describe cursor icon data info
MI_FB_CursorAttr_t	Describe mouse layer info
MI_FB_Rectangle_t	Describe the overlay display area
MI_FB_Cmap_t	Describe the color table info

Note: For data types not listed, please refer to mi_fb_datatype.h.

3.1. Standard Data Type

3.1.1 struct fb_bitfield

➤ Description

Bit field information for setting pixel format

➤ Definition

```
struct fb_bitfield
{
```

```

        _u32 offset;          /* beginning of bitfield */
        _u32 length;         /* length of bitfield */
        _u32 msb_right;      /* != 0: Most significant bit is right */
    };

```

➤ Member

Member	Description	Supportability
offset	Color component start bit	Supported
length	Length of the bit occupied by the color component	Supported
msb_right	Whether the bit on the right is the most significant bit.	Only 0 is supported. That is, the leftmost bit is the most significant bit.

➤ Related Data Type and Interface
N/A.

3.1.2 struct fb_var_screeninfo

➤ Description

Variable screen information

➤ Definition

```

struct fb_var_screeninfo
{
    _u32 xres;                /* visible resolution */
    _u32 yres;
    _u32 xres_virtual;        /* virtual resolution */
    _u32 yres_virtual;
    _u32 xoffset;             /* offset from virtual to visible */
    _u32 yoffset;             /* resolution */

    _u32 bits_per_pixel;      /* guess what */
    _u32 grayscale;          /* != 0 Graylevels instead of colors */

    struct fb_bitfield red;    /* bitfield in fb mem if true color, */ struct
    struct fb_bitfield green; /* else only length is significant */ struct
    struct fb_bitfield blue;
    struct fb_bitfield transp; /* transparency */

    _u32 nonstd;              /* != 0 Non standard pixel format */

```



```

_u32 activate;                /* see FB_ACTIVATE_* */

_u32 height;                 /* height of picture in mm */
_u32 width;                  /* width of picture in mm */

_u32 accel_flags;            /* (OBSOLETE) see fb_info.flags */

/* Timing: All values in pixclocks, except pixclock (of course) */
_u32 pixclock;               /* pixel clock in ps (pico seconds) */
_u32 left_margin;            /* time from sync to picture */
_u32 right_margin;           /* time from picture to sync */
_u32 upper_margin;           /* time from sync to picture */
_u32 lower_margin;
_u32 hsync_len;              /* length of horizontal sync */
_u32 vsync_len;              /* length of vertical sync */
_u32 sync;                   /* see FB_SYNC_* */
_u32 vmode;                  /* see FB_VMODE_* */
_u32 rotate;                 /* angle we rotate counter clockwise */
_u32 reserved[5];            /* Reserved for future compatibility */
};

```

➤ Member

Member	Description	Supportability
xres	Visible screen width (number of pixels)	Supported
yres	Visible screen height (number of pixels)	Supported
xres_virtual	Virtual screen width (image width in memory). When the value is smaller than xres, xres will be modified to be equal to that value.	Supported
yres_virtual	Virtual screen height (image height in memory). When the value is smaller than yres, yres will be modified to be equal to that value. If used in combination with xres_virtual, this parameter can pan image quickly in horizontal or vertical direction.	Supported
xoffset	Number of offset pixels in x direction	Supported, default is 0
yoffset	Number of offset pixels in y direction	Supported, default is 0
bits_per_pixel	Number of bits occupied by each pixel	Supported
grayscale	Grayscale	Not supported, default is 0 (color)
red	Red bit field information in the color	Supported

Member	Description	Supportability
	component	
green	Green bit field information in the color component	Supported
blue	Blue bit field information in the color component	Supported
transp	Alpha bit field information in the color component	Supported
nonstd	Indicates whether standard pixel format is used	Not supported, default is 0 (standard pixel format supported)
activate	Set the time of activation	Not supported, default is FB_ACTIVATE_NOW (activated immediately)
height	Screen height, unit is mm	Not supported, default is -1
width	Screen width, unit is mm	Not supported, default is -1
accel_flags	Acceleration flag	Not supported, default is -1
pixclock	Time required to show a pixel, unit is ns	Not supported, default is -1
left_margin	Represent the left blanking signal, the right blanking signal, and the horizontal synchronization duration. The sum of these values is equal to the horizontal retrace time in unit of dot clock.	Not supported, default is 64
right_margin		
hsync_len		
upper_margin	Represent the upper blanking signal, the lower blanking signal, and the vertical synchronization duration. The sum of these values is equal to the vertical retrace time in unit of dot clock.	Not supported, default is -1
lower_margin		
vsync_len		
sync	Signal synchronization method	Not supported, default is -1
vmode	Scan mode	Not supported, default is -1
rotate	Angle of rotation clockwise	Not supported, default is 0 (no rotation)

➤ Note

The default resolution of the HD device graphics layer is 1280x720; the default resolution of the SD device graphics layer is 720x576, and the cursor layer default resolution is 128x128. The pixel format is ARGB1555.

➤ Related Data Type and Interface

[struct fb_bitfield](#)
[FBIOGET_VSCREENINFO](#)
[FBIOPUT_VSCREENINFO](#)

3.1.3 struct fb_fix_screeninfo

➤ Description

Fixed screen information

➤ Definition

```
struct fb_fix_screeninfo
{
    char id[16]; /* identification string eg "TT Builtin" */ unsigned long
    smem_start; /* Start of frame buffer mem (physical
                address) */
    _u32 smem_len; /* Length of frame buffer mem */
    _u32 type; /* see FB_TYPE_ * */
    _u32 type_aux; /* Interleave for interleaved Planes */
    _u32 visual; /* see FB_VISUAL_ * */
    _u16 xpanstep; /* zero if no hardware panning */
    _u16 ypanstep; /* zero if no hardware panning */
    _u16 ywrapstep; /* zero if no hardware ywrap */
    _u32 line_length; /* length of a line in bytes */ unsigned long
    mmio_start; /* Start of Memory Mapped I/O (physical
                address) */
    _u32 mmio_len; /* Length of Memory Mapped I/O */
    _u32 accel; /* Indicate to driver which specific chip/card we have */
    _u16 reserved[3]; /* Reserved for future compatibility */
};
```

➤ Member

Member	Description	Supportability
id	Device driver name	Supported
smem_start	Memory start physical address	Supported
smem_len	Memory size	Supported
type	Graphics card type	Fixed to FB_TYPE_PACKED_PIXELS(p ixel values closely packed)
type_aux	Auxiliary type	Not supported. Undefined graphics card type under FB_TYPE_PACKED_PIXELS.
visual	Color mode	Not supported, default is FB_VISUAL_TRUECOLOR (true color)
xpanstep	Support horizontal PAN display or not: 0: Not supported ≠ 0: Supported. The value in this case is used to indicate the pixel	Fixed to 1

Member	Description	Supportability
	value per step in the horizontal direction	
ypanstep	Support vertical PAN display or not: 0: Not supported ≠ 0: Supported. The value in this case is used to indicate the pixel value per step in the vertical direction	Fixed to 1
ywrapstep	Similar to ypanstep, except that when it is displayed to the bottom, it can be returned to the beginning of the video memory for display.	Not supported, default is 0
line_length	Number of bytes per line	Supported
mmio_start	Memory map I/O start address	Not supported, default is 0
mmio_len	Memory map I/O length	Not supported, default is 0
accel	Indicate the hardware acceleration device supported	Not supported, default is FB_ACCEL_NONE (no acceleration)
reserved	Reserved	Not supported, default is 0

➤ Note

N/A.

➤ Related Data Type and Interface

[FBIOGET_FSCREENINFO](#)

3.1.4 struct fb_cmap_user

➤ Description

Describe the color table info

➤ Definition

```
struct fb_cmap_user
{
    u32 start;    /* First entry */
    u32 len; /* Number of entries */
    u32 __user *red; /* Red values */
    u32 __user *green;
    u32 __user *blue;
    u32 __user *transp; /* transparency, can be NULL */
};
```

➤ Member

Member	Description	Supportability
start	Entry of the first element	Supported
len	Number of elements	Supported
red	Red component start position	Supported
green	Green component start position	Supported
blue	Blue component start position	Supported
transp	Transparency component start position	Supported

➤ Note

N/A.

➤ Related Data Type and Interface

N/A.

3.2. MI_FB Data Type

3.2.1 MI_FB_BitField_t

➤ Description

Bit field information for setting pixel format.

➤ Related Data Type and Interface

[MI_FB_GetVarScreenInfo](#)

[MI_FB_SetVarScreenInfo](#)

3.2.2 MI_FB_VarScreenInfo_t

➤ Description

Variable screen information

➤ Related Data Type and Interface

[MI_FB_GetVarScreenInfo](#)

[MI_FB_SetVarScreenInfo](#)

3.2.3 MI_FB_FixScreenInfo_t

➤ Description

Fixed screen information

➤ Related Data Type and Interface

[MI_FB_GetFixScreenInfo](#)

3.2.4 MI_FB_ColorFmt_e

➤ Description

Collection of pixel formats supported by SigmaStar FB

➤ Definition

```
typedef enum
{
    E_MI_FB_COLOR_FMT_RGB565      = 1,
    E_MI_FB_COLOR_FMT_ARGB4444    = 2,
    E_MI_FB_COLOR_FMT_I8          = 4,
    E_MI_FB_COLOR_FMT_ARGB8888    = 5,
    E_MI_FB_COLOR_FMT_ARGB1555    = 6,
    E_MI_FB_COLOR_FMT_YUV422      = 9,
    E_MI_FB_COLOR_FMT_I4          = 13,
    E_MI_FB_COLOR_FMT_I2          = 14,
    /// Invalid color format.
    E_MI_FB_COLOR_FMT_INVALID
}MI_FB_ColorFmt_e;
```

➤ Member

Member	Description
E_MI_FB_COLOR_FMT_RGB565	RGB565 format
E_MI_FB_COLOR_FMT_ARGB4444	ARGB4444 format
E_MI_FB_COLOR_FMT_I8	I8
E_MI_FB_COLOR_FMT_ARGB8888	ARGB8888 format
E_MI_FB_COLOR_FMT_ARGB1555	ARGB1555 format
E_MI_FB_COLOR_FMT_YUV422	YUV422 format
E_MI_FB_COLOR_FMT_I4	I4
E_MI_FB_COLOR_FMT_I2	I2
E_MI_FB_COLOR_FMT_INVALID	Invalid pixel format

➤ Related Data Type and Interface

MI_FB_DisplayLayerAttr_t

3.2.5 MI_FB_OuputColorSpace_e

➤ Description

Color space output by FB overlay

➤ Definition

```
typedef enum
```

```
{
    E_MI_FB_OUTPUT_RGB = 0,
    E_MI_FB_OUTPUT_YUV = 1
}MI_FB_OutputClrSpace_e;
```

➤ Member

Member	Description
E_MI_FB_OUTPUT_RGB	RGB ColorSpace
E_MI_FB_OUTPUT_YUV	YUV ColorSpace

➤ Related Data Type and Interface

MI_FB_DisplayLayerAttr_t

3.2.6 MI_FB_DstDisplayPlane_e

➤ Description

The target module that FB overlay output to display chain, which can select the image adjustment process that the overlay participates in by configuring different display chain.

➤ Definition

```
typedef enum
{
    E_MI_FB_DST_IP0 = 0,
    E_MI_FB_DST_IP0_SUB = 1,
    E_MI_FB_DST_MIXER2VE = 2,
    E_MI_FB_DST_OP0 = 3,
    E_MI_FB_DST_VOP = 4,
    E_MI_FB_DST_IP1 = 5,
    E_MI_FB_DST_IP1_SUB = 6,
    E_MI_FB_DST_MIXER2OP = 7,
    E_MI_FB_DST_VOP_SUB = 8,
    E_MI_FB_DST_FRC = 9,
    E_MI_FB_DST_VE = 10,
    E_MI_FB_DST_BYPASS = 11,
    E_MI_FB_DST_OP1 = 12,
    E_MI_FB_DST_MIXER2OP1 = 13,
    E_MI_FB_DST_DIP = 14,
    E_MI_FB_DST_COPScaling = 15,
    E_MI_FB_DST_OP_DUAL_RATE = 16,
    E_MI_FB_DST_INVALID = 17,
}MI_FB_DstDisplayPlane_e
```

➤ Member

Member	Description
E_MI_FB_DST_OP0	OP0 path
E_MI_FB_DST_OP1	OP1 path

Note: Only supports E_MI_FB_DST_OP0/E_MI_FB_DST_OP1

- Related Data Type and Interface
MI_FB_DisplayLayerAttr_t

3.2.7 MI_FB_DisplayLayerAttrMaskBit_e

- Description
Specify the attributes to be updated in MI_FB_DisplayLayerAttr_t
- Definition

```
typedef enum
{
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS = 0x1,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE = 0x2,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_BUFFER_SIZE = 0x4,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SCREEN_SIZE = 0x8,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_PREMUL = 0x10,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_COLOR_FMB = 0x20,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_OUTPUT_COLORSPACE = 0x40,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_DST_DISP = 0x80,
}MI_FB_DisplayLayerAttrMaskBit_e
```
- Member

Member	Description
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS	Update display position
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE	Update display range
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_BUFFER_SIZE	Update buff size
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SCREEN_SIZE	Update screen resolution
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_PREMUL	Update premultiply alpha
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_COLOR_FMB	Update pixel format
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_OUTPUT_COLORSPACE	Update output color space
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_DST_DISP	Update the destination module of the overlay output to the display chain

- Related Data Type and Interface
MI_FB_DisplayLayerAttr_t

3.2.8 MI_FB_CursorAttrMaskBit_e

- Description
Specify the attributes to be updated in MI_FB_CursorAttr_t

- Definition

```
typedef enum
{
    E_MI_FB_CURSOR_ATTR_MASK_ICON = 0x1,
    E_MI_FB_CURSOR_ATTR_MASK_POS = 0x2,
    E_MI_FB_CURSOR_ATTR_MASK_ALPHA = 0x4,
    E_MI_FB_CURSOR_ATTR_MASK_SHOW = 0x8,
    E_MI_FB_CURSOR_ATTR_MASK_HIDE = 0x10,
    E_MI_FB_CURSOR_ATTR_MASK_COLORKEY = 0x20,
    E_MI_FB_CURSOR_ATTR_MASK = 0x3F,
}MI_FB_CursorAttrMaskBit_e
```

- Member

Member	Description
E_MI_FB_CURSOR_ATTR_MASK_ICON	Mouse icon changes
E_MI_FB_CURSOR_ATTR_MASK_POS	Mouse position changes
E_MI_FB_CURSOR_ATTR_MASK_ALPHA	Alpha changes
E_MI_FB_CURSOR_ATTR_MASK_SHOW	Show mouse
E_MI_FB_CURSOR_ATTR_MASK_HIDE	Hide mouse
E_MI_FB_CURSOR_ATTR_MASK_COLORKEY	Color key changes
E_MI_FB_CURSOR_ATTR_MASK	Identify all settable attributes of the mouse

- Related Data Type and Interface
MI_FB_CursorAttr_t

3.2.9 MI_FB_DisplayLayerAttr_t

- Description
Describe overlay attribute info

- Definition
typedef struct
{

```

MI_U32 u32XPos;      /**the x pos of origin point in screen*/
MI_U32 u32YPos;      /**the y pos of origin point in screen*/
MI_U32 u32DstWidth;   /**display buffer dest with in screen*/
MI_U32 u32DstHeight;  /**display buffer dest height in screen*/
MI_U32 u32DisplayWidth; /**the width of display buf in fb */
MI_U32 u32DisplayHeight; /**the height of display buf in fb. */
MI_U32 u32ScreenWidth; /**the width of screen */
MI_U32 u32ScreenHeight; /** the height of screen */
MI_BOOL bPreMul;      /**the data drawn in buffer whether is premultiply alpha or not*/
MI_FB_ColorFmt_e eFbColorFmt; /**the color format of framebuffer*/
MI_FB_OutputClrSpace_e eFbOutputColorSpace; /**output color space*/
MI_FB_DstDisplayPlane_e eFbDestDisplayPlane; /**destination displayplane*/
MI_U32 u32DisplayInfoMask; /** display attribute modify mask*/
}MI_FB_DisplayLayerAttr_t;

```

➤ Member

Member	Description	Supportability
offset	Color component start bit	Supported
u32XPos	The X coordinate of the layer at the origin of the screen.	Supported
u32YPos	The Y coordinate of the layer at the origin of the screen.	Supported
u32DstWidth	The display width of the layer on the screen	Supported
u32DstHeight	The display height of the layer on the screen	Supported
u32DisplayWidth	The width of the video memory resolution.	Supported
u32DisplayHeight	The height of the video memory resolution.	Supported
u32ScreenWidth	The width of the screen display resolution.	Supported
u32ScreenHeight	The height of the screen display resolution.	Supported
bPreMul	Whether the data in FB is premultiplied data.	Supported
eFbColorFmt	FB pixel format.	Supported
u32SetAttrMask	When setting the layer information, the parameter modifies the mask bit.	Supported
eFbOutputColorSpace	Set the Color space of the overlay output	Supported
eFbDestDisplayPlane	Set the destination module of the overlay output to the display chain.	Supported

Member	Description	Supportability
	The default is E_MI_FB_DST_OP0.	

➤ Related Data Type and Interface

[MI_FB_GetDisplayLayerAttr](#)

[MI_FB_SetDisplayLayerAttr](#)

3.2.10 MI_FB_GlobalAlpha_t

➤ Description

Alpha Information Structure

➤ Definition

```
typedef struct
{
    MI_BOOL bAlphaEnable;    /* alpha enable flag */
    MI_BOOL bAlphaChannel;   /* alpha channel enable flag */
    MI_BOOL bAlphaInvert;
    MI_U8 u8Alpha0;          /* alpha0 value */
    MI_U8 u8Alpha1;          /* alpha1 value */
    MI_U8 u8GlobalAlpha;     /* global alpha value */
    MI_U8 u8Reserved;
} MI_FB_GlobalAlpha_t;
```

➤ Member

Member	Description
bAlphaEnable	Enable Alpha overlay, the default is 1.
bAlphaChannel	Enable the Alpha channel, the default is 0.
bAlphaInvert	Alpha invert flag
u8Alpha0	Alpha0 value, the range is 0~255, the default is 255. In the RGB1:5:5:5 format, when the highest bit is 0, select this value as the Alpha superimposition value.
u8Alpha1	Alpha1 value, the range is 0~255, the default is 255. In the RGB1:5:5:5 format, when the highest bit is 1, select this value as the Alpha superimposition value.
u8GlobalAlpha	Global Alpha value, the range is 0~255, the default is 255. It takes effect when the Alpha channel is enabled.

➤ Related Data Type and Interface

[MI_FB_GetGlobalAlpha](#)

[MI_FB_SetGlobalAlpha](#)

3.2.11 MI_FB_ColorKey_t

➤ Description

Colorkey information structure

➤ Definition

```
typedef struct
{
    MI_BOOL bKeyEnable;
    MI_U8 u8Red;
    MI_U8 u8Green;
    MI_U8 u8Blue;
} MI_FB_ColorKey_t
```

➤ Member

Member	Description
bKeyEnable	Colorkey enable. TRUE: Enable FALSE: Not enable
u8Red	Colorkey R component value
u8Green	Colorkey G component value
u8Blue	Colorkey B component value

➤ Related Data Type and Interface

[MI_FB_GetColorKey](#)

[MI_FB_SetColorKey](#)

3.2.12 MI_FB_CursorImage_t

➤ Description

Describe cursor icon data info

➤ Definition

```
typedef struct
{
    MI_U32 u32Width;    /**width, unit pixel*/
    MI_U32 u32Height;   /**Height, unit pixel*/
    MI_U32 u32Pitch;    /**Pitch, unit pixel*/
    MI_FB_ColorFmt_e eColorFmt; /**Color format*/
    const char *pData;
}MI_FB_CursorImage_t;
```

➤ Member

Member	Description
u32Width	Icon width
u32Height	Icon height
u32Pitch	pitch
eColorFmt	Icon data pixel format
pData	Icon data

- Related Data Type and Interface
MI_FB_CursorAttr_t

3.2.13 MI_FB_CursorAttr_t

- Description
Describe mouse layer info

- Definition

```
typedef struct
{
    MI_U32 u32XPos;
    MI_U32 u32YPos;
    MI_U32 u32HotSpotX;
    MI_U32 u32HotSpotY;
    MI_FB_GlobalAlpha_t stAlpha;
    MI_FB_ColorKey_t stClrKey;
    MI_BOOL bShown;
    MI_FB_CursorImage_t stCursorImageInfo;
    MI_U16 u16CursorInfoMask;
}MI_FB_CursorAttr_t;
```

- Member

Member	Description
u32XPos	X coordinate of mouse position
u32YPos	Y coordinate of mouse position
u32HotSpotX	Mouse hotspot X coordinate
u32HotSpotY	Mouse hotspot Y coordinate
stAlpha	MI_FB_GlobalAlpha_t, mouse layer alpha info
stColorKey	MI_FB_ColorKey_t, mouse layer colorkey info
bShown	Whether the mouse is visible TRUE: visible FALSE: invisible
MI_FB_CursorImage_t	Icon data info.
u16CursorInfoMask	Set the mouse layer parameters to modify the mask bit

- Related Data Type and Interface
 - [MI_FB_GetCursorAttr](#)
 - [MI_FB_SetCursorAttr](#)

3.2.14 MI_FB_Rectangle_t

- Description

Describe the overlay display area

- Definition


```
typedef struct
{
    MI_U32 u32XPos;
    MI_U32 u32YPos;
    MI_U32 u32Width;
    MI_U32 u32Height;
}MI_FB_Rectangle_t;
```

- Member

Member	Description
u32XPos	X coordinate of upper left corner
u32YPos	Y coordinate of upper left corner
u32Width	Width
u32Height	Heigh

- Related Data Type and Interface
 - [MI_FB_GetScreenLocation](#)
 - [MI_FB_SetScreenLocation](#)

3.2.15 MI_FB_Cmap_t

- Description

Describe the color table info

- Related Data Type and Interface
 - [MI_FB_SetCmap](#)
 - [MI_FB_GetCmap](#)

4. ERROR CODE

Error Code	Macro Definition	Description
0x0	MI_SUCCESS	succeeded
0xA00B2011	MI_ERR_FB_INVALID_PARAM	Invalid parameter
0xA00B2006	MI_ERR_FB_NULL_PTR	Abnormal null pointer
0xA00B2001	MI_ERR_FB_INVALID_DEVID	Invalid device number
0xA00B2018	MI_ERR_FB_DRV_FAILED	Failure returned by FB driver
0xA00B2015	MI_ERR_FB_NOT_INIT	Uninitialized FB
0xA00B2008	MI_ERR_FB_NOT_SUPPORT	Not supported

5. PROCFS INTRODUCTION

5.1. cat

➤ Debug info

```
# cat /proc/mi_modules/mi_fb0
```

```
#####OnDumpOSDAttr#####
Framebuffer id = MStar FB0
xres=1280, yres=720
xres_virtual=1280, yres_virtual=1440
xoffset=0,yoffset=0
fix.line_length=0xa00 Bytes
fix.smem_start=0xf800000
Memory Size=0x400000 Bytes
Gop ID=0
Gwin ID=0
Open Count=1
Visible State=0
MIU Sel=0
ColorSpace=YUV
ColorFomrmat=ARGB1555
StretchWindow Pos[0,0]
StretchWindow Src[1280,720],StretchWindow Dst[1920,1080]
Gwin Pos[0,0]
Gwin Size[1280,720]
Gwin PhyAddr=0x0
ColorKey Enable=0
ColorKey Val=[0,0,0]
Enable Alpha Blend=1
Enalbe Multi Alpha=0
Global Alpha Val=0x0
Alpha0=0x0,Alpha1=0x0,(Only for ARGB1555)
GOP Hstart=192
Current TimingWidth=1920,TimingWidth=1080,hstar=192
#####end#####

#####OnDumpHwcursorAttr#####
Cursor Gop ID=4
Cursor Gwin ID=6
Cursor MIU Sel=0
Cursor PhyAddr=0x166000
Cursor ColorFmt=ARGB8888
Cursor Icon Width=44,Height=56
Cursor HotSpot[18,9]
Cursor request pos[712,207]
Cursor Visible=1
Cursor Gwin Pos[694,198]
```



```
Cursor Gwin Size[44,56]
Cursor Gwin Pitch=0x200 Bytes
Curosr StretchWindow pos[0,0]
Cursor StretchWin Src[1280,720],Dst[1920,1080]
Cursor ColorKey Enable=0
Cursor ColorKey Value=[0xff,0xff,0xff]
#####OnDumpHwcursorAttr#####
```

➤ Debug info analysis

Record the current Fbdev and Hwcursor usage status and OSD layer/OSD device/Hwcursor attributes, and can dynamically obtain information, which is convenient for debugging and testing.

➤ Parameter Description

Parameter		Description
OSD info	Framebuffer id	Framebuffer device ID
	xres	Visible resolution width
	yres	Visible resolution height
	xres_virtual	Virtual resolution width
	yres_virtual	Virtual resolution height
	xoffset	X offset from visible resolution to virtual resolution
	yoffset	Y offset from visible resolution to virtual resolution
	fix.line_length	Bytes per line of virtual resolution
	fix.smem_start	Start address in frame buffer
	Memory Size	Frame buffer memory size, unit: byte
OSD device info	Gop ID	Hardware ID of the layer
	Gwin ID	ID of the hardware display window
	Open Count	Open count of this layer. Add 1 when the user calls open(); minus 1 when calling close()
	Visible State	Display status of this layer. Value: 1: visible; 0: invisible. The user calls FBIOSET_SHOW to set the layer state.
	MIU Sel	The physical memory number for storing frame buffer data. For MSR620 devices, the value is 0.
	ColorSpace	The color space of the layer output. Value: YUV: Output YUV color space, default value; RGB: Output RGB color space.
	ColorFomrmat	Layer color format. Value: ARGB1555, ARGB4444, ARGB8888, RGB565. Update after the user sets the format item in the variable screen.
	StretchWindow Pos	The starting position of the layer on display device. Unit: pixel, default is 0, user can call

Parameter		Description
		FBIOSET_SCREEN_LOCATION to set it
	StretchWindow Src	The width and height of the original resolution of the layer. Unit: pixel. Use the hardware zoom function to zoom in to the current timing. Update after the user sets the visible resolution in the variable screen.
	StretchWindow Dst	The width and height of the layer on the display device. By default it is set to output timing size. Set by FBIOSET_SCREEN_LOCATION.
	Gwin Pos	The starting position of the layer display window. Unit: pixel, for OSD, the value is always (0,0)
	Gwin Size	The width and height of the layer display window. Unit: pixel. For OSD, this value is always set to the width and height of the visible resolution. Update after the user sets the visible resolution in the variable screen.
	Gwin PhyAddr	The physical address of the layer currently displayed data
	ColorKey Enable	Enable colorKey or not. Value: 0: N; 1: Y; User can set it by FBIOSET_COLORKEY
	ColorKey Val	RED/GREEN/BLUE channels of ColorKey. Value range of each channel: 0-255. ColorKey val value is in RGB888 format. Users can update it after setting FBIOSET_COLORKEY.
	Enable Alpha Blend	Enable Alpha Blend or not. Value: 0: N; 1: Y, default. Users can update it after setting FBIOSET_GLOBAL_ALPHA. Pixel alpha config is invalid when disabled it. Only pixel alpha is valid when it is enabled and Multi Alpha is disabled. (For ARGB1555 format, Alpha bit is 1, using Global Alpha as pixel alpha; Alpha bit is 0, using 0xff as pixel alpha.) When this item and Multi Alpha are enabled, the pixel alpha and Global Alpha are both valid.
	Enable Multi Alpha	Enable Multi Alpha or not. Value: 0: Y; 1: N, default value. Enable Multi Alpha, then Global Alpha are valid.
	Global Alpha Val	Global Alpha
	Alpha0	In ARGB1555 format, when the highest bit is 0, select Alpha0 as the Alpha blending value. Value: 0~255, the default is 0.
	Alpha1	In ARGB1555 format, when the highest bit is 1, select Alpha1 as the Alpha blending value. Value: 0~255, the default is 0.

Parameter		Description
	GOP Hstart	The offset of the layer hardware to the display window. This value is related to the current output timing. When the output timing changes, the layer driver is notified by FBIOSET_DISPLAYLAYER_ATTRIBUTES and the u32SetAttrMask value of MI_FB_DisplayLayerAttr_s contains the E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE mask bit. The layer driver will set the corresponding hstart value according to the current timing
	Current TimingWidth,	The width and height of the current output timing and the offset relative to the display window that should be set for the layer hardware under the current timing. Used in conjunction with GOP Hstart to check whether the hstart value set to the hardware layer is consistent with the current timing.
	Current TimingHeight,	
	hstar	
Hwcursor info	Cursor Gop ID	Cursor layer hardware ID
	Cursor Gwin ID	Cursor layer hardware display window ID
	Cursor MIU Sel	The physical memory number for storing cursor icon data. For MSR620 devices, the value is 0.
	Cursor PhyAddr	The physical address of the cursor layer currently displayed data
	Cursor ColorFmt	The cursor layer format. Value: ARGB1555,ARGB4444,ARGB8888,RGB565. User can set it by FBIOSET_CURSOR_ATTRIBUTE
	Cursor Icon Width	The width and height of the cursor icon. Unit: pixel, user can set it by FBIOSET_CURSOR_ATTRIBUTE.
	Cursor HotSpot	Cursor icon hotspot information. Unit: pixel. Hotspot information is usually related to the cursor icon, and user can set it by FBIOSET_CURSOR_ATTRIBUTE
	Cursor request pos	Cursor request position. Unit: pixel. User can set it by FBIOSET_CURSOR_ATTRIBUTE.
	Cursor Visible	Cursor visible or not. Value: 0: N; 1: Y. User can set it by FBIOSET_CURSOR_ATTRIBUTE.
	Cursor Gwin Pos	Cursor Gwin starting position. Unit: pixel.
	Cursor Gwin Pitch	Bytes per line of the cursor layer. The value is fixed at 0x200.
	Curosr StretchWindow pos	Cursor layer starting position on the display device. Unit: pixel.
	Cursor StretchWin Src	Original size of cursor layer. Unit: pixel. This value is usually the same as the OSD display

Parameter		Description
		resolution.
	Cursor StretchWin Dst	The width and height of the cursor layer on the display device. The value is the same as the output timing.
	Cursor ColorKey Enable	Cursor ColorKey enable or not. Value: 0: N; 1: Y. User can set it by FBIOSET_CURSOR_ATTRIBUTE.
	Cursor ColorKey Value	RED/GREEN/BLUE channels of cursor colorKey. Value range of each channel: 0-255. ColorKey val value is in RGB888 format. Users can update after setting FBIOSET_CURSOR_ATTRIBUTE.

5.2. echo

Function	Turn on or off the specified layer
Command	echo GUI_SHOW [FbID] [Status] > /proc/mi_modules/mi_fb0
Parameter Description	[FbID] index of framebuffer devices.
	[Status] on / off
Example	echo GUI_SHOW 0 on > /proc/mi_modules/mi_fb0 Turn on the layer fb0
	echo GUI_SHOW 0 off > /proc/mi_modules/mi_fb0 Turn off the layer fb0

Function	Turn on or off the HW cursor
Command	echo CURSOR_SHOW [FbID] [Status] > /proc/mi_modules/mi_fb0
Parameter Description	[FbID] index of framebuffer devices.
	[Status] on / off
Example	echo CURSOR_SHOW 0 on > /proc/mi_modules/mi_fb0 Turn on the cursor
	echo CURSOR_SHOW 0 off > /proc/mi_modules/mi_fb0 Turn off the cursor

Function	Set Colorkey for the specified layer
Command	echo GUI_SET_CLRKEY [FbID] [Red] [Green] [Blue] > /proc/mi_modules/mi_fb0
Parameter Description	[FbID] index of framebuffer devices.
	[Red] Red component, hex , in ARGB888 format.
	[Green] Green component, hex, in ARGB888 format.

	[Blue] Blue component, hex, in ARGB8888 format.
Example	echo GUI_SET_CLRKEY 0 ff 00 00 > /proc/mi_modules/mi_fb0 Set the Colorkey of fb0 to red.

Function	Dump the current displayed cursor icon
Command	echo CURSOR_DUMP [FbID] [Path] >/proc/mi_modules/mi_fb0
Parameter Description	[FbID] index of framebuffer devices. [Path] Path to save dump data
Example	echo CURSOR_DUMP [LayerID] /mnt > /proc/mi_modules/mi_fb0 Dump cursor icon to /mnt, the file name is CursorData.raw, open with 7yuv, set the width and height to 128, and display Rawdata according to the cursor format setting.

Function	Set the pixel alpha value for alpha bit0 and alpha bi1 of ARGB1555 format respectively. Because currently msr620 does not support this feature, it has not been implemented yet
Command	echo GUI_SETALPHA_ARGB1555 [FbID] [Alpha0] [Alpha1]> /proc/mi_modules/mi_fb0
Parameter Description	[FbID] index of framebuffer devices. [Alpha0] alpha0 value [Alpha1] alpha1 value
Example	None